



React Advanced

Wie zijn wij



Robbe Bierebeeck



Wouter Heirstrate

House rules

Wie zijn jullie

Ons doel

Onze aanpak

**De komende week bestaat uit zowel
lessen als zelfstandig werken**

De volgende lessen staan op het programma

1. Typescript
2. React Architectuur
3. State management
4. React Query
5. React Router
6. i18n in React

Typescript

**Typescript, de belangrijkste tool in
je toolbox**

In de praktijk maken we vooral gebruik van **Typescript**, in tegenstelling tot plain Javascript.

Typescript vormt een **laag bovenop** Javascript. Je kan dus in Typescript gewoon plain Javascript schrijven.

Typescript dient als een **belangrijke eerste verdedigingslinie** bij het schrijven van code.

We kijken naar een live-voorbeeld

Voorbeeld 1

**Onze code is correct,
maar toch krijgen we
niet het gewenste
resultaat.**

We moeten een manier hebben om zeker te zijn dat we dit soort bugs niet meer kunnen tegenkomen.

We willen aan een variabele kunnen zeggen dat het van een **bepaald type** is, zodat de code ons duidelijk aangeeft dat we de mist in gaan.

Typescript staat ons toe om
variabelen te typen zodat we onze
intentie kunnen duidelijk maken.

We kijken naar een live-voorbeeld

Voorbeeld

**Doordat we
aanduiden dat name
een **string array**
moet zijn, kunnen we
die niet meer
assignen met een
string**

Typing zorgt voor **stabiliteit** in de code.
We weten altijd wat het type van een
bepaalde variable zal zijn en kunnen
daar geen andere data aan toewijzen.

Op die manier kunnen we als developer
onze **intentie** duidelijk maken aan de
developers die later aan de slag gaan
met onze code, maar ook onszelf in de
(nabije) toekomst.

Er bestaan een
aantal **basis types**
die we kunnen
gebruiken.

- string
- boolean
- number
- Array
- Set
- ...

Base types zijn handig, maar in de praktijk hebben we nood aan **meer complexe types**.

**Zaken als een API
response zijn zelden
een base type, maar
vaak een object met
properties.**

Om dit te gaan opvangen kunnen we onze
eigen types gaan aanmaken, dit doen we
aan de hand van **interfaces**.

We kijken naar een live-voorbeeld

Voorbeeld

Properties in een interface kunnen bestaan uit base types, maar ook custom types.

Door interfaces op te stellen kunnen we onszelf verzekeren dat we enkel de properties die we nodig hebben en terugkrijgen van de API.

We stellen als het ware een **contract** op met onszelf en toekomstige developers dat de data waar we mee werken altijd op dezelfde manier eruit zal zien.

Soms maken we interfaces die maar
heel weinig van elkaar afwijken.

We kijken naar een live-voorbeeld

Voorbeeld

Via **overerving** kunnen we interfaces verder uitbreiden

Aan de hand van de extends functie kunnen we bestaande interfaces uitbreiden en **zo stukken code die we eerder geschreven hebben opnieuw gebruiken.**

Code die we niet twee keer schrijven is code waar maar op een plaats een bug kan in kruipen.

Soms maken we interfaces waar we
stukken **variabel** willen maken.

We kijken naar een live-voorbeeld

Voorbeeld

**We kunnen dus
types en interfaces
doorgeven via de <>
notatie!**

We kunnen zoveel types meegeven
zoals we willen.

**Let hierbij op de naam die je aan je
generic types geeft.** Vaak worden
generic types gewoon “T” genoemd,
maar dit is vaak niet voldoende
duidelijk wat het doel van de generic
type is.

Wanneer zouden we generics concreet gebruiken?

We gebruiken generics wanneer we in heel verschillende data toch een herbruikbare structuur kunnen vrijwaren.

Hierdoor kunnen we de **herbruikbare structuur abstraheren** naar een apart stukje code.

Deze code is compacter, gemakkelijker uit te breiden, en gemakkelijker te onderhouden.

Soms willen we bepaalde properties van een interface optioneel maken

Optionele types zorgen ervoor dat bepaalde properties al dan niet gedefinieerd moeten zijn.

Hierdoor kunnen we **relatief onbelangrijke** properties weglaten uit een object.

Let hierbij zeker op dat je geen kritieke logica voor de UI optioneel maakt.

React Architecture

**Werken met components is de basis
van modern webdevelopment**

**Om component
based te kunnen
werken, moeten we
weten hoe, wanneer
en waarom we
components maken.**

Veel startende developers weten dat ze component based moeten werken, maar weten maar weinig waarom ze dat doen, wanneer ze dat moeten doen, en hoe ze dat best gestructureerd doen.

We werken component based om
onzelf en toekomstige developers
te helpen.

**Door onze code in
kleine components
op te delen, zijn deze
makkelijker te
onderhouden.**

Kleine components bevatten weinig logica. In **weinig logica zit weinig code**, en in weinig code kan je makkelijker bugs spotten of zelfs vermijden.

Door onze code in kleine components op te delen, kunnen we deze gemakkelijker hergebruiken.

Componenten die een specifieke functie hebben kunnen we gemakkelijker gaan abstraheren, zodat ze gemakkelijk kunnen **ingezet worden op verschillende plaatsen.**

Zo kunnen we bijvoorbeeld een button component voorzien die overal bruikbaar is, zodat we deze niet steeds opnieuw moeten schrijven.

**Beslissen of we iets
herbruikbaar maken
doen we aan de hand
van het 3-6-9
principe.**

Wanneer we zeker zijn dat we het binnen de drie maand opnieuw kunnen hergebruiken, **maken we het component herbruikbaar.**

Wanneer we weten dat we binnen de 6 maand het component kunnen hergebruiken, kijken we naar de **effort om het herbruikbaar te maken.**

Wanneer we we weten dat we binnen de 9 maand mogelijks het component kunnen hergebruiken, maken we deze **nog niet herbruikbaar.**

Concreet denken we in termen van features, core en shared.

Sommige onderdelen van een applicatie zien we als een feature, bv een pagina om aan te melden. We maken voor **elke feature een folder** aan waarin we onze componenten bijhouden.

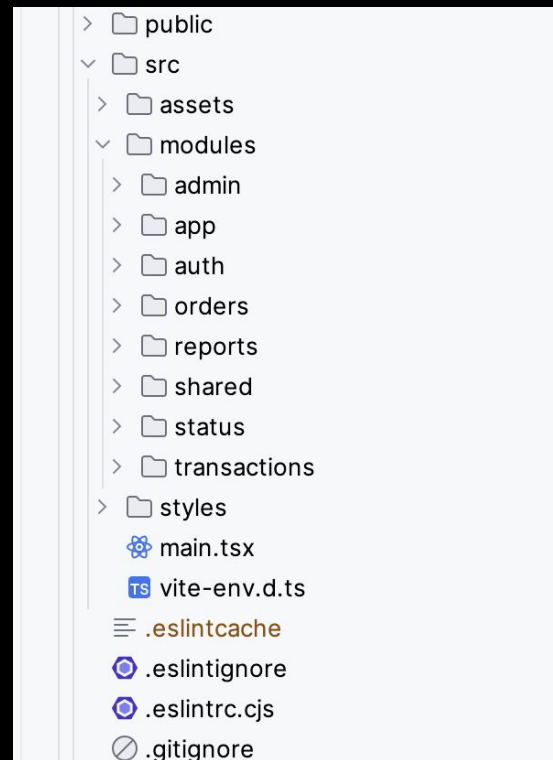
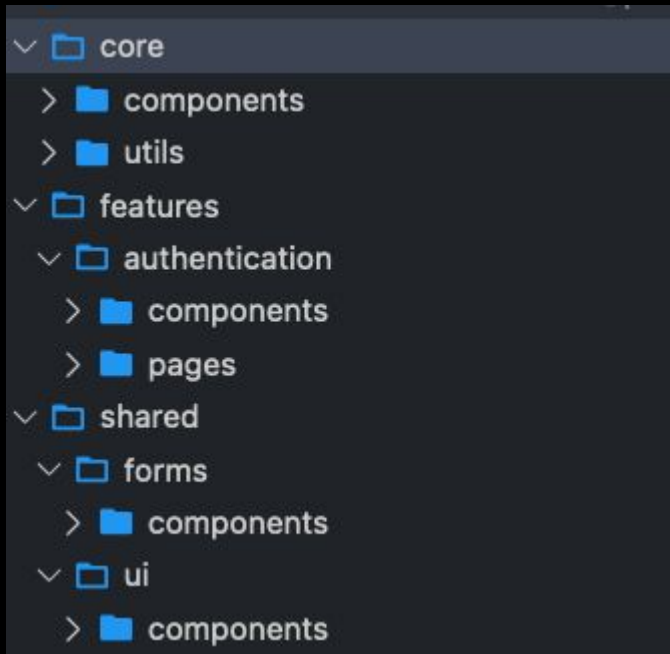
Andere onderdelen van de applicatie zijn essentieel om de applicatie te gebruiken. Deze houden we bij in de **core folder**.

Andere onderdelen zijn overal inzetbaar, maar zijn niet verplicht. Deze onderdelen steken we in de **shared folder**.

**Binnen een feature
maken een
onderscheid tussen
componenten en
paginas.**

Een pagina is een fysieke pagina waar een **gebruiker kan op belanden** binnen de applicatie. Deze bestaat vaak uit kleinere componenten.

Componenten zijn de **bouwstenen waarmee we een pagina opstellen.**



Sommige onderdelen van de applicatie zien we als **shared**, waaronder bijvoorbeeld **custom hooks**

Wat zijn **hooks**?

React Hooks zijn speciale functies die je toelaten om **state** en andere React-functionaliteiten te gebruiken in **functionele componenten**.

Hooks zoals **useState** en **useEffect** zijn standaard beschikbaar, maar je kan ook eigen hooks schrijven om logica te delen tussen componenten zond

Wanneer **custom hook** schrijven?

Je schrijft een **custom hook** wanneer je een stuk logica hebt dat je **herhaaldelijk gebruikt in meerdere componenten** bijvoorbeeld data ophalen, event listeners, of complex statebeheer.

In plaats van die logica telkens opnieuw te schrijven, verpak je ze in een eigen hook. Zo hou je je code **korter, leesbaarder en beter onderhoudbaar**

```
1  import { useRouter } from 'next/router';
2
3  import { Locale } from '@shared/utils/i18n';
4
5  export const useLocale :() => Locale = (): Locale => { Show usages  Bert Verhelst
6      const router :NextRouter = useRouter();
7      return (router.locale || Locale.nl) as Locale;
8  }
9
```

Bonus: extensies en tips

1. Error Lens: inline errors op de error-lijn
2. Meerdere lijnen in- en uitcommentaar:

Preferences > Open Keyboard Shortcuts >

`editor.action.commentLine`